

Gymnázium J. K. Tyla



**Praktický dopad AI na softwarové
inženýrství - AI agenti a agentické
inženýrství**

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce zkoumá přechod od konverzační asistence k delegovanému agentnímu vývoji a roli harnessu jako běhového a integračního prostředí coding agenta. Cílem je vysvětlit, jak propojení jazykového modelu s nástroji, stavem, verifikací a lidskými kontrolními body umožňuje řízené provádění softwarových úloh v praxi. Agentické inženýrství práce vymezuje jako soubor postupů, které činí AI-asistovaný vývoj účinným, kontrolovaným, opakovatelným a škálovatelným. Teoretická část vychází z odborných publikací a dokumentace nástrojů. Praktická část analyzuje systém DarkFactory prostřednictvím statické inspekce workflow, runneru, kontejnerového prostředí, deklarativního grafu a testů v přesně určené revizi [e9c10221](#). Analýza dokládá fázované řízení požadavku, schvalovací brány, izolaci změn, deterministickou verifikaci, seberevizi, ukládání checkpointů a obnovu přerušného běhu. Výsledky ukazují, že praktická autonomie nevzniká pouze schopnostmi modelu, ale především návrhem harnessu a přesným rozdělením odpovědnosti mezi deterministické mechanismy a lidské rozhodování. Dostupná evidence však neměří kvalitu práce různých modelů ani provozní úspěšnost systému. Závěry jsou proto omezeny na strukturální vlastnosti této jediné reprodukovatelně pinované revize.

Klíčová slova: jazykové modely; coding agents; harness; Agentické inženýrství; GitHub Actions; DarkFactory

Abstract

This thesis examines the transition from conversational assistance to delegated agentic software development and the role of the harness as a coding agent's execution and integration environment. Its objective is to explain how connecting a language model to tools, persistent state, verification, and human control points enables governed execution of software tasks. Agentic Engineering is defined as the set of practices that makes AI-assisted development effective, controlled, repeatable, and scalable. The theoretical part draws on research literature and first-party tool documentation. The practical part analyses DarkFactory through static inspection of the workflow, runner, container environment, declarative graph, and tests at the exact [e9c10221](#) revision. The analysis identifies staged request handling, approval gates, change isolation, deterministic verification, self-review, checkpoint persistence, and interrupted-run recovery. The findings indicate that practical autonomy does not arise from model capability alone, but primarily from harness design and an explicit division of responsibility between deterministic mechanisms and human decisions. The evidence does not measure model quality or operational success rates. The conclusions are therefore limited to structural properties of this single reproducibly pinned revision.

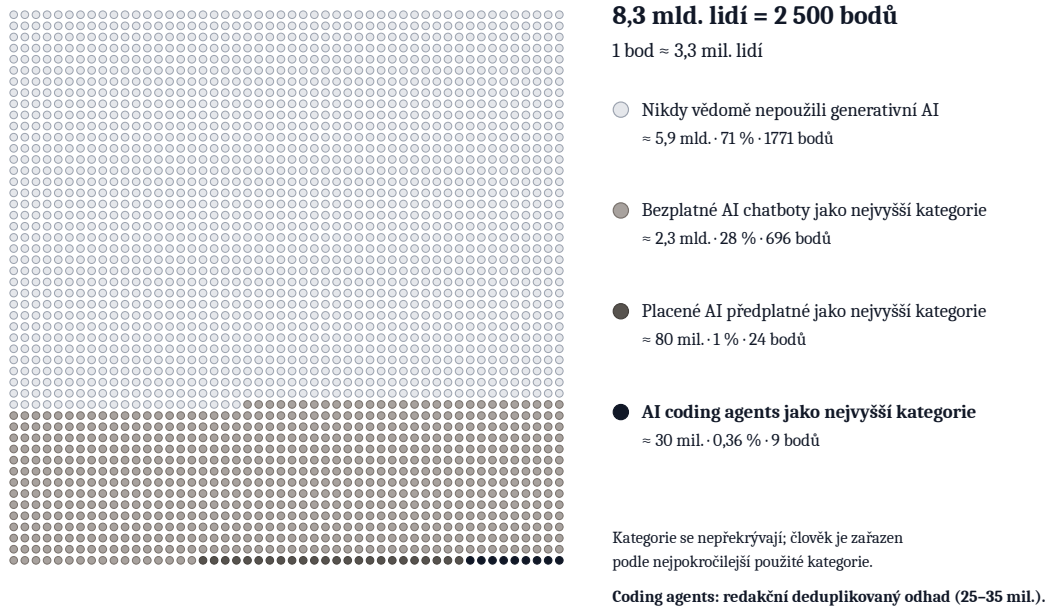
Keywords: language models; coding agents; harness; Agentic Engineering; GitHub Actions; DarkFactory

Obsah

Prohlášení	2
1 Úvod	5
1.1 Cíl a vymezení	5
2 Teoretická část	6
2.1 Jazykový model v agentním systému	6
2.1.1 Context window a kompakce	6
2.2 Agent a harness	7
2.2.1 Smyčka	7
2.3 Agentické inženýrství	8
2.3.1 Zadání a kontext	8
2.3.2 Orchestrace a lidská integrace	8
3 Praktická část	11
3.1 Metodika	11
3.2 DarkFactory	12
3.2.1 Spuštění a kontejnerizované prostředí	12
3.2.2 Fázovaný průchod a řídicí smyčka	13
3.2.3 Serializace stavu, checkpointing a obnova	13
4 Výsledky a diskuse	14
4.1 Zjištění	14
4.2 Diskuse	15
5 Závěr	16
Seznam zdrojů	17
Seznam obrázků a tabulek	19

1 Úvod

Nástroje založené na velkých jazykových modelech prošly rychlým vývojem: od doplňování kódu při psaní přes konverzační chatboty až po autonomní agenty, kteří pomocí nástrojů samostatně provádějí změny a spouštějí příkazy v běhovém prostředí (1; 2; 3; 4; 5). S rostoucími schopnostmi modelů roste i jejich adopce, avšak pravidelné využívání plnohodnotných agentických systémů zůstává omezeno na přibližně 0,36 % světové populace (6). Odhad zastoupení jednotlivých kategorií uživatelů shrnuje Obrázek 1.



Obrázek 1: Odhad rozdělení uživatelů generativní AI. Pravidelní uživatelé agentů pro tvorbu kódu tvoří přibližně 0,36 % světové populace, tedy zhruba 30 milionů lidí (6).

Aby mohl agent samostatně pracovat na projektu, nestačí pouhé generování odpovědí. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění testů a CI, uchování stavu mezi jednotlivými kroky a vymezený bod, v němž člověk rozhodne o přijetí výsledku (7; 8).

1.1 Cíl a vymezení

Cílem práce je ukázat, jak harness dělá z jazykového modelu autonomního agenta a jaké postupy umožňují využívat agenty účinně a kontrolovaně.

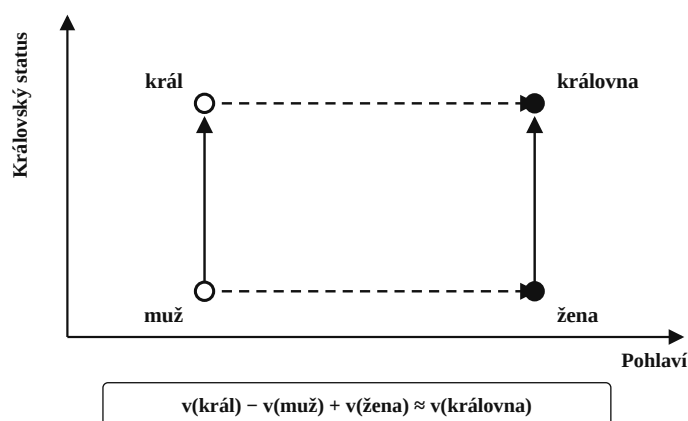
Praktická část analyzuje DarkFactory, na poskytovateli nezávislou pipeline pro AI-asistovaný softwarový vývoj. Hodnocená revize kombinuje workflow GitHub Actions, kontejnerizovaný pythonovský řadič a harness `df` s deklarativním grafovým jádrem v TypeScriptu.

2 Teoretická část

2.1 Jazykový model v agentním systému

Jazykový model (LLM) předpovídá další token na základě **kontextu**. **Transformer** zpracovává vztahy mezi nimi pomocí mechanismu attention (9; 10). Při inferenci model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů. Jednotlivé volání však samo nevybírá další kontext, nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje a jejich výsledky vrací do dalšího kroku (11).

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy ve vektorovém prostoru. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (12). Tento vztah schematicky znázorňuje Obrázek 2.



Obrázek 2: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **král**, **královna**, **muž** a **žena** (12).

2.1.1 Context window a kompakce

Kontextové okno (context window) tvoří pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat (13). Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (14).

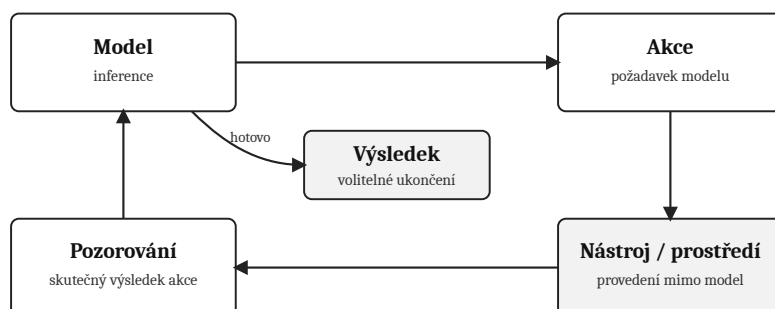
Kompakce po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce (14).

2.2 Agent a harness

Zásadní rozdíl mezi konverzačním chatbotem a autonomním agentem nespočívá v architektuře použitého modelu, nýbrž v míře delegace provádění. Chatbot setrvává v roli externího rádce: uživatel musí manuálně kopírovat úryvky kódu, dodávat kontext a spouštět navržené příkazy. Agent naproti tomu prostřednictvím harnessu získává přímý přístup k nástrojům repozitáře. Sám prochází souborovou strukturu, upravuje kód, spouští testy a na základě chybových výstupů samostatně koriguje své změny (11; 15). Tento posun transformuje roli člověka z přímého vykonavatele na dohlížejícího architekta.

2.2.1 Smyčka

Základním mechanismem agentického systému je iterativní řídicí smyčka. Na rozdíl od jednorázové generace odpovědi u chatbota probíhá interakce v cyklu podle vzoru **ReAct** (**Reasoning and Acting**) (16). Model v každém kroku analyzuje aktuální stav kontextu, zformuluje vnitřní uvažování a navrhne konkrétní volání nástroje ve formě strukturovaného požadavku. Harness tuto akci provede v prostředí projektu, zachytí výsledek a vrátí jej modelu jako nové pozorování. Smyčka pokračuje, dokud model nedosáhne cíle nebo nenarazí na bezpečnostní limit kroků či tokenů; její průběh znázorňuje Obrázek 3.



Obrázek 3: Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku; princip podle (16).

2.3 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) představuje systematický návrh podmínek, v nichž může jazykový model vykonávat inženýrské úlohy účinně, kontrolovaně, opakovatelně a v souladu s pravidly projektu (7; 8). Zahrnuje dekompozici úloh, přesné ohraničení kontextu, konstrukci nástrojových rozhraní, verifikační zpětnou vazbu a mechanismy řízené integrace.

2.3.1 Zadání a kontext

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo **spec-driven development**, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku. **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (14; 17). Projektová pravidla a validační příkazy lze uchovat přímo v repozitáři například ve standardizovaném souboru `AGENTS.md`, takže je harness může připojit k relevantní úloze (18).

Specifikace a kontext se promítají do provádění prostřednictvím schopností harnessu. **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy; **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (19). **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu (20). **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (21). Nejde o samostatné agenty, ale o řízené části runtime, jejichž výběr, oprávnění a výstupy určují, co může agent skutečně provést a ověřit.

Po každé akci harness vrací agentovi pozorování, například výsledek testu, překladače nebo stav pracovního stromu. Agent podle něj upraví další krok a pokračuje, dokud nesplní podmínky přijetí nebo nenarazí na stanovený limit.

2.3.2 Orchestrace a lidská integrace

Rozsáhlou úlohu lze rozdělit do více agentních běhů, pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek. Nezávislé podúlohy mohou zpracovat **paralelní pracovníci**, zatímco **workflow graph** předem určuje závislosti, pořadí a větvení fází (22). **Agent Swarm** v systému Kimi K2.5

dynamicky rozkládá úlohu na heterogenní podproblémy a spouští specializované agenty paralelně pod řízením orchestrátoru (23).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol.

Goal loop tvoří nadřazenou řídicí smyčku vůči jednotlivým cyklům agentní smyčky: když model ve standardní smyčce ReAct označí svůj krok za dokončený, harness porovná dosažený stav s cílem a podmínkami přijetí a rozhodne, zda běh ukončit, nebo úlohu vrátit agentovi k další iteraci či změně strategie (7; 16).

Princip **human-in-the-loop (HITL)** doplňuje automatizované smyčky o kontrolní brány, v nichž je vyžadováno explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak nadále odpovídá za záměr, architekturu a integraci, zatímco agent provádí ohraničenou implementační práci. Začlenění změn do hlavní větve vyžaduje vedle technického ověření v CI také lidskou revizi a převzetí odpovědnosti za výsledek (24; 25).

PRAKTICKÁ ČÁST

DarkFactory:

**Pipeline pro automatizované
AI-asistované softwarové inženýrství**

3 Praktická část

3.1 Metodika

Praktická část navazuje na teoretická východiska o harnessu a Agentickém inženýrství. Zkoumá, jak jsou principy řízeného provádění, ohraničení kontextu a verifikace realizovány ve zdrojovém kódu DarkFactory. Jediným referenčním bodem je commit `e9c10221b40589512d262a0edb95f709b923150c`; submodul, evidenční manifest i všechny praktické citace odkazují na tuto revizi (26).

Analýza se zaměřuje na čtyři klíčové inženýrské dimenze odvozené z teoretické části:

1. **Izolace běhového prostředí a oprávnění:** mechanismus hermetického oddělení agenta od hostitelského CI systému, montování pracovního stromu repozitáře a injektování autentizačních tajemství s minimálními právy.
2. **Dekompozice úlohy a fázované řízení (HITL):** rozdělení životního cyklu požadavku do diskrétních stavů (interpretace, plánování, implementace) a začlenění formálních schvalovacích bran člověkem.
3. **Deterministická verifikace a seberevize:** integrace automatizovaných nástrojů (lintery, testovací frameworky), záchyt chybových kódů v samoopravné smyčce a auditování vygenerovaného diffu vůči schválenému plánu.
4. **Perzistence stavu a zotavení:** serializace parametrů běhu (checkpointing), záchyt výpadků kvót externích API (HTTP 429) a procedura obnovení rozpracované úlohy příkazem `/resume`.

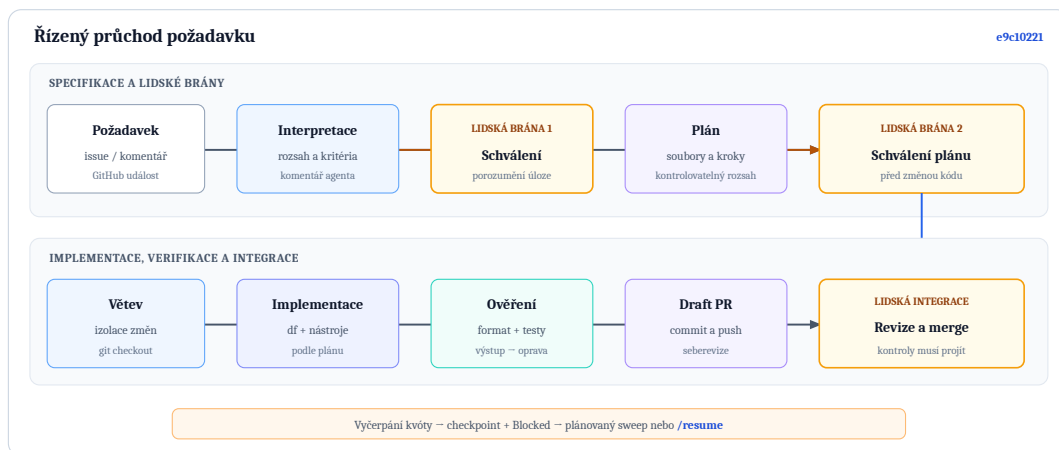
Metodou je trasovatelná statická inspekce workflow `.github/workflows/agent.yml`, obrazu `docker/Dockerfile.agent`, produkčního řadiče `.github/scripts/agent_runner.py`, grafu `harness/assets/graph.darkfactory.json`, jeho ukládání stavu v `harness/src/graph/run-state.ts` a souvisejících testů. Zjištění popisují mechanismy, které jsou v revizi přítomné a testované; nepředstavují měření provozní úspěšnosti ani srovnání kvality modelů.

Revize obsahuje dvě související, ale nezaměnitelné vrstvy orchestrace. Produkční workflow vstupuje do pythonovského řadiče, který volá příkaz `df run`. Vedle něj je v repozitáři deklarativní grafové jádro v TypeScriptu s uzly, hranami, bezpečnostními rozpočty smyček a atomicky ukládaným stavem `df`. Práce proto neprezentuje grafové jádro jako vstupní bod produkčního workflow, ale hodnotí jej jako samostatnou architektonickou součást stejného snapshotu (26).

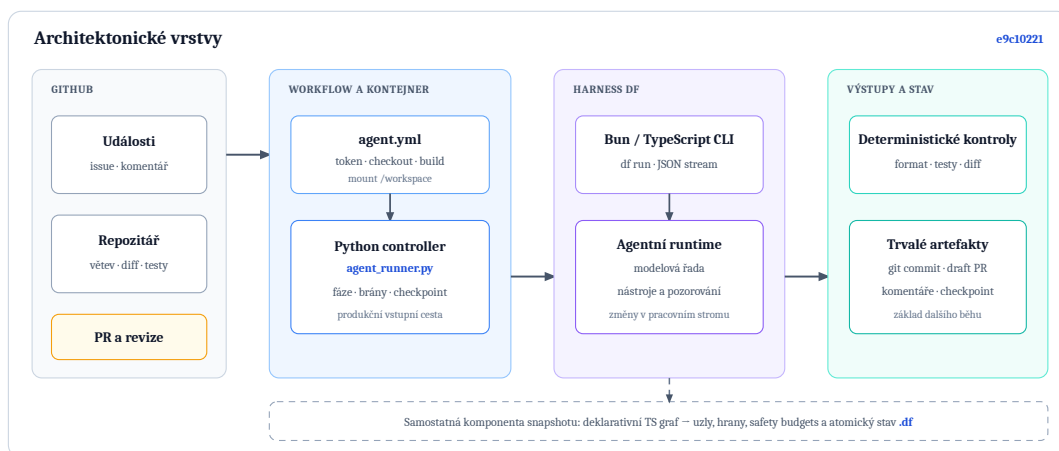
3.2 DarkFactory

DarkFactory představuje automatizovanou pipeline běžící v prostředí GitHub Actions. Workflow spouští kontejner Docker, v němž pythonovský runner připraví prostředí, předá harnessu instrukce a zpracuje výsledek jeho běhu (26).

Produkční průchod požadavku shrnuje Obrázek 4 a vztah workflow, řadiče, harnessu a odděleného grafového jádra znázorňuje Obrázek 5.



Obrázek 4: Produkční průchod požadavku v hodnocené revizi DarkFactory [e9c10221]: lidské brány oddělují interpretaci, plán a integraci výsledku (26).



Obrázek 5: Architektonické vrstvy hodnocené revize: produkční GitHub Actions/Python cesta používá harness df; deklarativní grafové jádro v TypeScriptu je v témže snapshotu přítomno jako samostatná komponenta (26).

3.2.1 Spuštění a kontejnerizované prostředí

Celý životní cyklus úlohy je vázán na webhooksy repozitáře. Pipeline je spuštěna událostmi otevření issue nebo přidání komentáře (`issues.opened`, `issue_comment.created`). Úvodní krok workflow v GitHub Actions nejprve ověřuje oprávnění autora události — odmítá vstupy generované automatickými boty a vyžaduje oprávnění k zápisu do repozitáře. Po validaci vstupu workflow sestaví izolovaný obraz Docker na základě předpisu `docker/Dockerfile.agent`. Tento obraz obsahuje základní systémové nástroje, interpret

jazyka Python, verzovací systém git a potřebné vývojové knihovny. Pracovní kopie repozitáře je do kontejneru připojena jako dedikovaný svazek (volume mount). Tím je zaručeno, že agent pracuje výhradně nad ohraničeným pracovním stromem a nemá přímý přístup k hostitelskému systému CI runneru. Přístupové tokeny GitHub API i klíče k externím modelům jsou injektovány do kontejneru jako přísně ohraničené proměnné prostředí s minimálními nezbytnými oprávněními (26).

3.2.2 Fázovaný průchod a řídicí smyčka

Řízení úlohy v kontejneru přebírá skript `agent_runner.py`. Průchod není realizován jako jediná dlouhá interakce, nýbrž je rozdělen do fází oddělených kontrolními branami:

- **Fáze interpretace:** Runner načte text issue a předloží jej modelu se systémovou instrukcí pro dekompozici problému. Model zformuluje pochopení úlohy a navrhne akceptační kritéria. Runner výsledek zapíše jako komentář k issue a ukončí běh, čímž vyčká na schválení člověkem.
- **Fáze plánování:** Po explicitním schválení interpretace je workflow spuštěno znovu. Model analyzuje strukturu repozitáře a sestaví konkrétní plán úprav — specifikuje seznam souborů určených ke změně a předpokládaný sled kroků. Tento plán je opět předložen člověku ke schválení.
- **Implementační smyčka a verifikace:** Teprve po schválení plánu runner vytvoří dedikovanou pracovní větev, aby nezasahoval do hlavní vývojové větve. Harness následně model navádí k úpravě kódu. Po dokončení úprav runner automaticky spustí deterministické ověřovací nástroje: linter a sadu jednotkových testů. V případě detekce syntaktické chyby či pádu testu má agent k dispozici přesný chybový výstup a jeden vyhrazený krok na samoopravu.
- **Seberevize a draft pull request:** Jakmile testy projdou, runner vytvoří commit, odešle větev na vzdálený server a otevře draft pull request. Následně proběhne fáze seberevize: runner vygeneruje diff mezi pracovní větví a výchozím stavem a porovná změněné soubory se schváleným rozsahem plánu. Pokud model či kontrolní skript odhalí změny mimo povolený rozsah, vyžádá se náprava. Při shodě je pull request označen jako připravený k lidské revizi (26).

3.2.3 Serializace stavu, checkpointing a obnova

V distribuovaném prostředí automatizovaných pipeline představují limity zdrojů (GitHub Actions timeout, kvóty externích poskytovatelů LLM či síťové výpadky) zásadní výzvu pro spolehlivost. DarkFactory tento problém řeší explicitní serializací stavu běhu (checkpointing).

Během agentních fází runner ukládá strukturovaný checkpoint s identifikátorem požadavku, větví a přehledem dokončených kroků. Při vyčerpání kvóty uchová stav, označí úlohu jako blokovanou a vypočte možnost obnovení; plánovaný sweep nebo příkaz `/resume` ná-

sledně znovu odešle běh. Grafové jádro odděleně ukládá aktuální uzel a výstupy atomickou náhradou souboru `.df`. Oba mechanismy v hodnocené revizi zajišťují perzistenci, nelze je však vydávat za jeden společný formát stavu (26).

4 Výsledky a diskuse

4.1 Zjištění

Pipeline organizuje průchod požadavku jako sled událostí v repozitáři. Po vytvoření issue runner vygeneruje interpretaci zadání; její schválení uživatelem iniciuje tvorbu plánu a teprve další schválení zahájí samotnou implementaci. Dvě schvalovací brány tak oddělují porozumění požadavku a plánování od samotných úprav zdrojového kódu (26).

Workflow před spuštěním ověřuje, zda je agent pro daný repozitář povolen, a ignoruje komentáře vytvořené automatickými boty. Autentizační tokeny jsou předány z prostředí GitHub Actions a pracovní strom projektu je připojen do kontejneru, v němž běží pythonový runner (26).

Po schválení plánu pipeline vytvoří pracovní větev a předá implementační instrukci harnessu. Runner spustí formátovací a testovací nástroje; při neúspěchu testů provede opravný krok. Poté vytvoří commit, odešle větev a otevře draft pull request. Seberevize porovná změněné soubory s rozsahem plánu; pokud detekuje nežádoucí úpravy mimo schválený rozsah, iniciuje opravu a novou revizi. Jestliže seberevize nenajde nesoulad, pipeline označí pull request jako připravený k lidské revizi (26).

Inspekce testovací sady ukazuje pokrytí invariantů kolem ukládání stavu, plánování grafu, detekce odchylek a předávání chybového výstupu do opravného kroku. Přítomnost těchto testů dokládá záměr deterministicky ověřovat řídicí mechanismy; sama o sobě však nepro- kazuje, že model vždy odstraní lokální regresi bez lidského zásahu (26).

Runner ukládá checkpoint pro spolehlivé navázání rozpracovaného běhu. Při vyčerpání kvóty uchová stav, označí běh jako blokový a umožní jeho pokračování příkazem `/ resume`. Testy pokrývají serializaci stavu, obnovení pipeline, přepnutí poskytovatele modelu i kontrolu přístupových oprávnění (26).

4.2 Diskuse

Pinovaný snapshot dokládá mechanismy trvalého stavu, oprávnění, izolace změn, automatických kontrol a obnovy. Neměří však účinnost jednotlivých postupů Agentického inženýrství, kvalitu práce s kontextem ani provozní úspěšnost orchestrace. Následující interpretace jsou proto omezeny na strukturální vlastnosti revize [e9c10221](#).

Dělbá odpovědnosti mezi modelem a harnessem. Jazykový model sám o sobě postrádá pojem o čase, kauzalitě i stavu vývojového prostředí; funguje jako stochastický generátor návrhů. Výsledky ukazují, že skutečnou páteř autonomního systému tvoří deterministický harness — v tomto případě GitHub Actions workflow a pythonovský runner. Právě harness zodpovídá za přípravu izolovaného prostředí, vynucování kroků, orámování kontextu a interpretaci návratových kódů. Model tedy nepředstavuje samostatnou autonomní entitu, nýbrž výpočetní modul zasazený do přísně strukturovaného algoritmického rámce (7).

Architektonický kompromis: Autonomie vs. správa. DarkFactory volí konzervativní architekturu se čtyřmi kontrolními branami — dvěma lidskými (schválení interpretace a plánu) a dvěma automatizovanými (testy a audit diffu). Tento přístup eliminuje nejčastější rizika nekontrolovaných agentů: drift zadání, neautorizované zásahy do konfigurací a nekonečné cykly generování kódu. Cenou za vysokou kontrolu je však fragmentace autonomie a vznik lidského úzkého hrdla. Proces nepředstavuje bezobslužný vývojový systém, nýbrž asistovanou výrobní linku, kde člověk plní roli architekta a revizora (8).

Ekonomika kontextu a perzistence stavu. V souladu s poznatky o degradaci pozornosti při dlouhém kontextu (13) DarkFactory neudrží historii celého projektu uvnitř context window. Verzovací systém git a externí checkpointy slouží jako primární nositelé stavu. Do context window vstupují při každém kroku pouze minimální nezbytné informace: zadání, schválený plán a bezprostřední pozorování z testů. Oddělení trvalého stavu od dočasného kontextu inference snižuje náklady na tokeny a udržuje pozornost modelu zaměřenou na řešený fragment kódu.

Deterministická verifikace jako protiváha stochastické generace. Softwarové inženýrství nabízí pro nasazení agentů zásadní výhodu: možnost strojově ověřitelné zpětné vazby překladačů, linterů a jednotkových testů. Analyzovaný runner předává oprávněmu kroku konkrétní výstup a současně jej omezuje schváleným plánem. Tato konstrukce vytváří podmínky pro cílenou opravu; spolehlivost samotné opravy by však musela prokázat samostatná provozní evaluace (26).

5 Závěr

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (7; 11). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů Agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Analýza pipeline DarkFactory doložila, jak lze takový systém realizovat v praxi: řídí průchod požadavku, vytváří izolovanou větev, ověřuje výsledek testy, kontroluje rozsah změn, ukládá checkpoint a předává výsledek k lidské revizi.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního psaní kódu k preciznímu návrhu zadání, vymezení mantinelů a určení bodů, v nichž je nezbytné lidské rozhodnutí. Zjištění ukazují, že samotná autonomie agenta nenahrazuje řízení vývojového procesu. Správně navržený harness spolu s postupy Agentického inženýrství umožňuje delegovat netriviální vývojové úlohy na stochastické modely a současně chránit stabilitu hlavní vývojové větve i transparentnost provedených změn. Rozsah této práce však neumožňuje zobecnit výsledky na jiné harnessy ani posoudit kvalitu kódu vytvářeného různými modely.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>
7. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
8. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
9. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
10. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>
11. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
12. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>

13. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
14. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
15. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
16. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
17. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
18. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
19. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
20. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
21. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
22. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
23. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>
24. GITHUB. *Branches | GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/reference/branches>
25. GITHUB. *About pull requests | GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
26. MARIUS, Patrik. *DarkFactory commit e9c10221* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>

Seznam obrázků a tabulek

Obrázek 1	Odhad rozdělení uživatelů generativní AI. Pravidelní uživatelé agentů pro tvorbu kódu tvoří přibližně 0,36 % světové populace, tedy zhruba 30 milionů lidí (6).....	5
Obrázek 2	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov král , královna , muž a žena (12).....	6
Obrázek 3	Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku; princip podle (16).	7
Obrázek 4	Produkční průchod požadavku v hodnocené revizi DarkFactory <code>e9c10221</code> : lidské brány oddělují interpretaci, plán a integraci výsledku (26).....	12
Obrázek 5	Architektonické vrstvy hodnocené revize: produkční GitHub Actions/Python cesta používá harness <code>df</code> ; deklarativní grafové jádro v TypeScriptu je v témže snapshotu přítomno jako samostatná komponenta (26).....	12